

Executive Summary

This report documents the Stage 16.1 verification audit of the ATHENA-X repository. The audit covered **221** declared components across three layers: **191 plugins** (indicators, patterns, options, cross-market, news, forecast), **14 engines**, and **16 data providers**. The verification process was strictly non-destructive: no source code was modified, no plugin was rewritten, and no architectural changes were made. Every conclusion in this report is backed by objective evidence gathered by automated static analysis, dynamic import probing, AST extraction, and live module instantiation.

The headline result is that **208 of 221 components (94%) are FAILED**, **13 (5%) are PROVISIONAL**, and **0 (0%) are VERIFIED**. Zero components reached VERIFIED status because the VERIFIED tier requires passing functional verification, trading-logic verification, integration verification, performance verification, AND historical validation. Of these five gates, historical validation is universally blocked because no plugin currently returns a real computed output — every indicator and pattern plugin raises *NotImplementedError* when its `compute()` method is invoked, and 165 of the 191 plugin slots have no source code at all.

PLUGINS

191 FAILED · 0 PROVISIONAL · 0 VERIFIED · 191 total

ENGINES

6 FAILED · 8 PROVISIONAL · 0 VERIFIED · 14 total

PROVIDERS

11 FAILED · 5 PROVISIONAL · 0 VERIFIED · 16 total

Most critical finding. The entire technical-analysis plugin layer is unimplemented. All 14 indicator plugins (EMA, RSI, MACD, SMA, VWAP, Bollinger, ADX, ATR, CCI, Ichimoku, Stochastic, Williams %R, OBV, Fibonacci) and all 6 pattern plugins (Candlestick, Wyckoff, Chan Theory, Elliott Wave, Smart Money Concepts, Volume Profile) ship as scaffolding stubs whose `compute()` methods raise *NotImplementedError*("STEP 4 implementation"). Additionally, no market-structure plugin exists, meaning HH/HL/LH/LL/BOS/CHOCH/Liquidity-Sweep detection — the foundation layer of the entire Stage 7 architecture — has no implementation.

Secondary finding. The plugin loader in `engines/plugin-engine/manager.py` looks for an entrypoint file named `indicator.py` containing a class named `EmaIndicator`, `EmaAgent`, or `EMAIndicator`. The actual indicator plugins use the filename `plugin.py` and class name `EmaPlugin`. Even if the `compute()` bodies were implemented, the loader would fail to find them via the primary code path and would fall back to a permissive "any class" heuristic — meaning the integration contract between the plugin-engine and the plugin contracts is currently broken.

Tertiary finding. The base plugin protocol `TechnicalIndicator` in `plugins/indicators/_base/protocol.py` declares `compute(input_data: IndicatorInput, params: IndicatorParams) -> IndicatorOutput` using structured dataclasses. Every indicator plugin's actual `compute` signature is `compute(inputs: dict, params: dict) -> dict`. The contract between the protocol and the implementations is therefore violated — even if `compute` bodies were filled in, callers expecting `IndicatorOutput` would receive a `dict` and have to re-parse.

Table of Contents

Executive Summary	1
Table of Contents	2
1. Complete Plugin Inventory	4
1.1 Inventory Methodology	4
1.2 Plugin Layer — Category Breakdown	4
1.3 Engine Layer — Inventory	4
1.4 Provider Layer — Inventory	5
2. Dependency Graph	7
2.1 Declared Edges	7
2.2 Implicit (Undeclared) Dependencies	7
2.3 Duplicated Functionality	7
3. Functional Verification	9
3.1 Verification Protocol	9
3.2 Plugin Layer Results	9
3.3 Compute() Signature Mismatch	9
3.4 Loader ↔ Plugin Filename Mismatch	9
3.5 Engine Layer Results	10
3.6 Provider Layer Results	10
4. Trading Logic Verification	11
4.1 Verification Result	11
4.2 Per-Plugin Logic Verification Matrix	11
5. Cross-Plugin Consistency Analysis	13
5.1 Consistency Hypotheses (For Future Validation)	13
6. Historical Validation	14
6.1 Quantitative Metrics (All Null)	14
7. Integration Verification	15
7.1 End-to-End Latency (Stages 1–5 Only)	15
8. Performance Verification	16
8.1 Top 10 Slowest Plugin Imports	16
8.2 Performance Budget vs Actual	16
9. Bugs Discovered	17
10. Root Cause Analysis	21

10.1 Root Cause #1 — Scaffold-First Development Without Implementation Phase	21
10.2 Root Cause #2 — Contract Drift Between Loader, Protocol, and Plugin Templates	21
10.3 Root Cause #3 — Documentation Tracking Slot Count Instead of Function Count	21
10.4 Causal Chain	21
11. Recommended Fixes	23
12. Certification Status	25
12.1 Aggregate Certification	25
12.2 Plugin-Layer Certification by Category	25
12.3 Engine-Layer Certification	25
12.4 Provider-Layer Certification	26
12.5 Final Certification Verdict	27

1. Complete Plugin Inventory

1.1 Inventory Methodology

The discovery phase performed a recursive filesystem walk of `/home/z/my-project/athena-x/` restricted to the three component roots: `plugins/`, `engines/`, and `providers/`. Each immediate subdirectory of these roots was treated as one component slot. For every slot, the discovery routine checked for the presence of: `manifest.yaml`, `manifest.py`, `pyproject.toml`, a `src/` tree (the canonical Python implementation location), a `tests/` directory, and a `README.md`. The total source line count, presence of stub markers (`NotImplementedError`, "STEP 4 implementation", "Implementation comes in STEP 4", "TODO: implement"), and a fallback heuristic (class declaration with <25 total LoC) were recorded for every slot.

1.2 Plugin Layer — Category Breakdown

Category	Total	Has src/	Stubs	No src/	Avg LoC
cross-market	89	0	89	89	0
forecast	9	0	9	9	0
indicators	14	14	14	0	38
news	10	0	10	10	0
options	63	6	63	57	3
patterns	6	6	6	0	33
TOTAL	191	26	191	165	—

The plugin layer is the largest layer by slot count (191 slots) but the smallest by real implementation. Out of 191 slots, only 26 carry any source code at all (14 indicators + 6 patterns + 6 options plugins), and all 26 of those source-bearing plugins are scaffolding stubs whose `compute()` methods raise `NotImplementedError`. The remaining 165 slots have manifest metadata, README, `pyproject.toml`, and tests scaffolding, but no `src/` directory and therefore no Python code whatsoever.

1.3 Engine Layer — Inventory

Engine	LoC	Stub	Tests LoC	Public API (sample)	Cert
ai-runtime	14	YES	2	<code>AiRuntimeEngine()</code>	FAILED
backtest-engine	14	YES	2	<code>BacktestEngineEngine()</code>	FAILED
cross-market-plugin-engine	293	no	140	<code>CorrelationMatrix();</code> <code>CorrelationEngine(update_returns,</code> <code>compute_correlation, compute_mat...</code>	PROVISIONAL
data-engine	14	YES	2	<code>DataEngineEngine()</code>	FAILED
forecast-engine	476	no	206	<code>EnsembleConsensus(combine);</code> <code>ExplainabilityEngine(explain)</code>	PROVISIONAL
governance-engine	580	no	176	<code>AuditTrail(record, get_entries, count);</code> <code>ConfidenceArbitrationEngine(arbitrate)</code>	PROVISIONAL
learning-engine	14	YES	2	<code>LearningEngineEngine()</code>	FAILED
narrative-engine	552	no	246	<code>EventClassifier(classify);</code> <code>MarketImpactEngine(assess_impact)</code>	PROVISIONAL
onnx-runtime	14	YES	2	<code>OnnxRuntimeEngine()</code>	FAILED

options-plugin-engine	181	no	66	OptionsPluginExecutor(execute, get_stats); OptionsPluginManager(registry, dependency_re...	PROVISIONAL
plugin-engine	1010	YES	282	PluginConfigService(load_from_file, load_from_dict, set_enabled, is_enabled, get_config...	PROVISIONAL
report-engine	14	YES	2	ReportEngineEngine()	FAILED
trade-engine	542	no	218	TradeQualificationEngine(qualify); OpportunityRankingEngine(rank)	PROVISIONAL
validation-framework	346	no	111	TestOrchestrator(run_all); TestPhase()	PROVISIONAL

8 of 14 engines carry real implementation totaling approximately 3,978 lines of Python: cross-market-plugin-engine (293 LoC, correlation + leadership), forecast-engine (476 LoC, ensemble + explainability + fusion), governance-engine (580 LoC, audit trail + self-healing + health supervisor), narrative-engine (552 LoC, event classifier + impact + generator), options-plugin-engine (181 LoC, manager + executor), plugin-engine (1,010 LoC, registry + dependency + scheduler + manager + config), trade-engine (542 LoC, qualification + ranking + risk), and validation-framework (346 LoC, orchestrator). These 8 engines represent the genuine institutional logic of the platform.

6 of 14 engines are scaffolding-only: ai-runtime, backtest-engine, data-engine, learning-engine, onnx-runtime, and report-engine. Each contains a single 12-line Python file whose sole content is a class declaration with a docstring reading "Implementation comes in STEP 4." These engines cannot perform their declared function — backtesting, GPU model serving, ONNX browser runtime, multi-format report generation, or self-correction learning are all unavailable.

1.4 Provider Layer — Inventory

Provider	LoC	Stub	Adapter Class	Tests	Cert
alphavantage	27	YES	AlphavantageAdapter	empty	FAILED
cnbc	26	YES	CnbcAdapter	empty	FAILED
cnn	126	no	CNNAdapter	yes (77L)	PROVISIONAL
databento	26	YES	DatabentoAdapter	empty	FAILED
failover	123	no	—	yes (104L)	PROVISIONAL
finnhub	143	no	FinnhubAdapter	yes (58L)	PROVISIONAL
flashalpha	27	YES	FlashalphaAdapter	empty	FAILED
fred	27	YES	FredAdapter	empty	FAILED
polygon	27	YES	PolygonAdapter	empty	FAILED
polymarket	26	YES	PolymarketAdapter	empty	FAILED
reuters	26	YES	ReutersAdapter	empty	FAILED
sec	26	YES	SecAdapter	empty	FAILED
simulated	122	no	SimulatedAdapter	yes (62L)	PROVISIONAL
trading-economics	26	YES	TradingEconomicsAdapter	empty	FAILED
wsj	26	YES	WsjAdapter	empty	FAILED
yahoo	176	no	YahooAdapter	yes (121L)	PROVISIONAL

5 of 16 providers are functional adapters: Yahoo Finance (176 LoC), Finnhub (143 LoC), CNN Business (126 LoC, Fear & Greed + RSS news), Simulated (122 LoC, deterministic test fixture), and Failover router (123 LoC). Each of these five has a real adapter class, real HTTP client setup, real response parsing, and an associated test file. They are PROVISIONAL rather than VERIFIED because live-network validation across an 8-hour trading session has not yet been completed for any of them.

11 of 16 providers are scaffolding-only: alphavantage, cnbc, databento, flashalpha, fred, polygon, polymarket, reuters, sec, trading-economics, wsj. Each is approximately 26 LoC containing a class declaration with no methods. The provider layer is therefore ~31% implemented by slot count but only ~5 of the 11 market data sources required for institutional coverage are reachable.

2. Dependency Graph

The dependency graph was constructed from the dependencies field declared in each plugin's manifest.yaml. The graph contains **191 nodes** and **13 directed edges**. **0 cycles were detected**. **0 plugins declared dependencies on non-existent plugins**.

2.1 Declared Edges

Plugin	Depends On
Odte_dealer_hedging	gex
bollinger	sma
dealer_delta	delta
dealer_gamma	gamma
dealer_hedging_pressure	gex
gamma_acceleration	gamma
gamma_flip	gex
gex	gamma
intraday_risk	gex, gamma_flip
macd	ema
negative_gamma_zone	gex
positive_gamma_zone	gex

All 13 declared edges are within the options-plugin layer and form a clean DAG (no cycles). The hub dependencies are gex (Gamma Exposure, depended on by 4 plugins), gamma (depended on by 3), and delta (depended on by 1). No cross-layer dependencies were declared — i.e., no indicator depends on another indicator, no pattern depends on an indicator, and no engine depends on a plugin. The manifest-level dependency graph is therefore **structurally clean but functionally underutilised**: the platform's actual runtime dependencies (e.g., MACD depends on EMA at the algorithm level, Wyckoff depends on market structure at the algorithm level) are not encoded in manifests.

2.2 Implicit (Undeclared) Dependencies

Beyond the 13 manifest-declared edges, the platform has substantial **implicit dependencies** that the manifest layer does not capture. These include: (a) every plugin implicitly depends on the plugins/indicators/_base/protocol.py TechnicalIndicator contract; (b) every options plugin implicitly depends on an option-chain input that no provider currently returns (Yahoo Finance returns OHLCV only); (c) the PluginManager implicitly depends on a class-naming convention that the actual plugins do not follow; (d) every forecast model implicitly depends on feature inputs that no feature-engineering module currently computes; (e) every DNA object implicitly depends on a contributor-model registry that is not wired to the plugin registry.

2.3 Duplicated Functionality

Several plugin slots appear to declare overlapping or duplicated functionality. Within the options layer: gamma-exposure and gex are almost certainly the same metric (Gamma Exposure); volatility-surface and

vol_surface are duplicate surface plugins; iv, iv_rank, iv_percentile, and iv_crush_probability all derive from the same underlying implied-volatility input but are declared as four separate plugins rather than one parameterised plugin. Within the cross-market layer: vix and vvix; vix_confirming and vix_not_confirming; spy_es_corr and spy_es_divergence; qqq_leads_spy and nvda_leads_qqq; us2y, us30y, and tn timer all represent overlapping treasury-yield monitors. The duplication is not a logic defect but inflates the slot count and creates maintenance surface.

3. Functional Verification

3.1 Verification Protocol

For every plugin slot, the verifier performed the following ordered checks: (1) presence of `src/` directory; (2) presence of `plugin.py` or `indicator.py` entrypoint file inside the `src` tree; (3) successful module import via `importlib.util.spec_from_file_location`; (4) successful class instantiation via `cls()`; (5) presence of `compute()` or `calculate()` method on the instance; (6) successful invocation of `compute({}, {})` with empty input dict and empty params dict (the minimum contract); (7) signature introspection via `inspect.signature`. Each check recorded elapsed time and any exception class + message.

3.2 Plugin Layer Results

Category	Slots	Has src	Imports OK	Instantiates OK	<code>compute()</code> callable
cross-market	89	0	0	0	0
forecast	9	0	0	0	0
indicators	14	14	14	14	0
news	10	0	0	0	0
options	63	6	6	6	0
patterns	6	6	6	6	0

Across the 26 source-bearing plugins, 100% successfully imported and instantiated. This confirms that the manifest metadata is consistent with the file layout — every plugin file parses as valid Python, every plugin module exposes a class, and every class accepts a no-argument constructor. However, **0% of source-bearing plugins have a callable `compute()` method**. All 26 raise `NotImplementedError("STEP 4 implementation")` when `compute()` is invoked. The remaining 165 plugin slots cannot even reach the `compute()` check because they have no source code to import.

3.3 `Compute()` Signature Mismatch

All 20 indicator + pattern plugins expose the same `compute()` signature: `compute(inputs: dict, params: dict) -> dict`. This signature is **inconsistent** with the base `TechnicalIndicator` Protocol declared in `plugins/indicators/_base/protocol.py`, which requires `compute(input_data: IndicatorInput, params: IndicatorParams | None = None) -> IndicatorOutput`. The Protocol uses structured dataclasses (`IndicatorInput` with `opens/highs/lows/closes/volumes/timestamps` fields; `IndicatorOutput` with `values/signals/metadata` fields); the plugins accept and return untyped dicts. Even if the plugin `compute` bodies were filled in, the plugin-engine would have to choose between: (a) marshalling `IndicatorInput` → dict before calling `compute`, then dict → `IndicatorOutput` after, losing type safety; or (b) violating the Protocol contract by calling `compute` with raw dicts and accepting a raw dict back. Neither option is currently encoded in the executor.

3.4 Loader ↔ Plugin Filename Mismatch

The `PluginManager.load()` method in `engines/plugin-engine/manager.py` (lines 117–129) attempts to locate the plugin entrypoint via two paths: first as an installed Python package `athena_x_plugin_indicators_{id}`; then as a file named `indicator.py` at either

plugin_path/src/{module_name}/indicator.py or plugin_path/indicator.py. The actual plugins use the filename plugin.py (never indicator.py). The loader would therefore fall through to the package-import path, which only works if the plugin has been pip installed into the environment — and no plugin is currently installed.

Even if the loader found the file, it searches for a class named {PluginId.title()}Indicator (e.g., EmaIndicator), then {PluginId.title()}Agent, then {PLUGIN_ID.upper()}Indicator, then the literal Indicator. None of these match the actual class name EmaPlugin. The loader's final fallback ("any class not starting with _") would catch EmaPlugin, but only as a last resort — and only for indicators that already failed all four preferred names. This is a fragile integration contract that will break the moment a plugin legitimately defines multiple classes.

3.5 Engine Layer Results

Engine verification used the same protocol adapted for engine entrypoints. The 8 substantial engines (≥100 LoC) all expose real classes with real methods: e.g., PluginRegistry.register/unregister/get/list_all/count, DependencyGraph.topological_sort/detect_cycles, SystemHealthSupervisor.update_subsystem/get_system_health/compute_readiness_score, TradeQualificationEngine.qualify, OpportunityRankingEngine.rank, RiskEngine.assess, TestOrchestrator.run_all. The 6 stub engines expose a single empty class whose body is a docstring; they cannot be exercised functionally.

3.6 Provider Layer Results

Provider verification confirmed that the 5 substantial adapters (Yahoo, Finnhub, CNN, Simulated, Failover) each expose a properly-named XxxAdapter class extending BaseProviderAdapter. Yahoo, Finnhub, CNN, and Simulated each declare async fetch_* methods and an async close() method. Yahoo additionally has the richest surface (176 LoC) with _fetch_ohlc, _fetch_quote, _fetch_vix, and a Yahoo-specific _determine_instrument_type helper. The 11 scaffolding providers expose only the adapter class declaration with no methods.

4. Trading Logic Verification

Trading-logic verification is the highest-priority phase. For each indicator and pattern plugin, the verifier attempted to evaluate `compute()` against known historical scenarios with pre-computed expected outputs. The test corpus was structured as follows:

- **EMA:** EMA20 > EMA50 > EMA200 with price > EMA20 → expected signal *Bullish Trend*.
- **RSI:** RSI = 75 → expected *Overbought*; RSI = 25 → expected *Oversold*; RSI higher-highs while price lower-highs → expected *Bullish Divergence*.
- **MACD:** MACD line crosses above signal line above zero → expected *Bullish Cross*; MACD crosses zero upward → expected *Zero-line Bullish Cross*; histogram expanding → expected *Momentum Expansion*.
- **VWAP:** Session reset at market open; price > VWAP + 2σ → expected *Upper Band Touch*; zero-volume instrument → expected *SKIP signal*.
- **Bollinger:** Price touches upper band → expected *Overbought*; band squeeze → expected *Low Volatility*.
- **Support/Resistance:** Pivot calculation on known OHLC → expected specific price levels.
- **Candlestick:** Doji, Hammer, Engulfing, Shooting Star, Morning Star, Evening Star, Three White Soldiers, Three Black Crows.
- **Wyckoff:** Phase detection (Accumulation phases A–E, Distribution phases A–E) on Spring/Upthrust events.
- **Chan Theory:** Trend, Segment, Central Zone (■■■), Buy/Sell Points (■■■/■■■/■■■/■■■/■■■/■■■), trend transition.
- **Market Structure:** HH, HL, LH, LL, BOS, CHOC, Liquidity Sweep.

4.1 Verification Result

TRADING-LOGIC VERIFICATION STATUS: BLOCKED

0 of 14 indicators and 0 of 6 patterns could be exercised. Every `compute()` method raises `NotImplementedError` before any computation can occur. The Market Structure plugin (HH/HL/LH/LL/BOS/CHOC/Liquidity Sweep) does not exist as a plugin slot at all. Trading-logic verification is therefore 100% blocked at the source — there is no logic to verify.

4.2 Per-Plugin Logic Verification Matrix

Plugin	Category	Test Cases	Status	Evidence
adx	indicators	Trend strength >25, <20, +DI/-DI crossover	BLOCKED	<code>compute()</code> raises <code>NotImplementedError</code>
atr	indicators	Volatility expansion/contraction	BLOCKED	<code>compute()</code> raises <code>NotImplementedError</code>
bollinger	indicators	Upper/lower touch, Squeeze, Bandwidth expansion	BLOCKED	<code>compute()</code> raises <code>NotImplementedError</code>
cci	indicators	Overbought >100, Oversold <-100, Zero-line cross	BLOCKED	<code>compute()</code> raises <code>NotImplementedError</code>
ema	indicators	Bullish stack (EMA20>50>200), Bearish stack, Price > EMA20	BLOCKED	<code>compute()</code> raises <code>NotImplementedError</code>
fibonacci	indicators	Retracement levels (38.2/50/61.8), Extension levels	BLOCKED	<code>compute()</code> raises <code>NotImplementedError</code>
ichimoku	indicators	Tenkan/Kijun cross, Cloud break, Chikou span	BLOCKED	<code>compute()</code> raises <code>NotImplementedError</code>

macd	indicators	Bullish cross, Bearish cross, Zero-line cross, Histogram expansion	BLOCKED	compute() raises NotImplementedError
obv	indicators	Trend confirmation, Divergence	BLOCKED	compute() raises NotImplementedError
rsi	indicators	Overbought (75), Oversold (25), Bullish divergence, Bearish divergence	BLOCKED	compute() raises NotImplementedError
sma	indicators	Bullish stack, Bearish stack, Golden cross, Death cross	BLOCKED	compute() raises NotImplementedError
stochastic	indicators	%K/%D cross, Overbought/Oversold, Divergence	BLOCKED	compute() raises NotImplementedError
vwap	indicators	Session reset, Deviation bands, Zero-volume indicator	BLOCKED	compute() raises NotImplementedError
williams-r	indicators	Overbought >-20, Oversold <-80	BLOCKED	compute() raises NotImplementedError
candlestick	patterns	Doji, Hammer, Engulfing, Shooting Star, Morning/Evening Star	BLOCKED	compute() raises NotImplementedError
chan-theory	patterns	Trend, Segment, Central Zone, Buy/Sell Points	BLOCKED	compute() raises NotImplementedError
elliott-wave	patterns	Impulse 1-2-3-4-5, Corrective A-B-C	BLOCKED	compute() raises NotImplementedError
smart-money	patterns	Order Block, Fair Value Gap, Break of Structure	BLOCKED	compute() raises NotImplementedError
volume-profile	patterns	POC, Value Area, High Volume Nodes	BLOCKED	compute() raises NotImplementedError
wyckoff	patterns	Accumulation A-E, Distribution A-E, Spring, Uptick	BLOCKED	compute() raises NotImplementedError

The 20 plugins above represent the complete set of plugins for which trading-logic verification would be meaningful. Each one's logic is documented in its README and manifest but is not enforced by any executable code. The market-structure plugin — the foundational layer per Stage 7 architecture — has no slot at all and must be created from scratch.

5. Cross-Plugin Consistency Analysis

Cross-plugin consistency was assessed across 191 plugins. Of these, **191 are stubs** and **0 have only scaffolding code**. The consistency matrix is therefore **not evaluable**: no plugin produces real signal output, so there is no plugin output to compare against another plugin's output. A meaningful consistency check would require, at minimum, that EMA, MACD, Market Structure, and Wyckoff all return bullish signals on the same uptrending input — but none of these plugins can return any signal at all.

Reason: Of 191 plugins, 191 are stubs and 0 have only scaffolding code. No plugin returns real signal output, so cross-plugin agreement cannot be measured.

5.1 Consistency Hypotheses (For Future Validation)

When implementations land, the following consistency hypotheses should be evaluated against historical SPY/ES 0DTE sessions:

- **Strong agreement:** EMA bullish stack + MACD bullish cross + Market Structure HH/HL + Wyckoff Phase C/D (accumulation) + Chan Theory ■■■/■■■ should all fire together on confirmed uptrends.
- **Justified disagreement:** RSI Overbought on a strong trend is NOT a sell signal — it is a continuation signal. RSI Overbought + Wyckoff Distribution Phase D would be a true disagreement worth flagging.
- **Unjustified disagreement:** Market Structure bullish (HH/HL) + Wyckoff Distribution would be a contradiction — one of the two plugins would have a logic bug.
- **Lag measurement:** BOS (Break of Structure) should fire before CHOCH (Change of Character) on trend reversals; CHOCH before Wyckoff Spring; Wyckoff Spring before Chan Theory ■■■.
- **Confirmation timing:** EMA20/50 cross should lag MACD signal-line cross by 1–3 candles on 15m timeframe; RSI divergence should lead both.

6. Historical Validation

HISTORICAL VALIDATION STATUS: BLOCKED

Historical replay requires plugin compute() implementations. As all 26 source-bearing plugins raise NotImplementedError and 165 plugins have no source at all, no replay is possible.

Recommended action: Implement plugin compute() bodies first, then replay historical SPY/ES ODTE sessions through the engine pipeline.

6.1 Quantitative Metrics (All Null)

Metric	Value
Detection Accuracy	N/A — blocked
False Positives	N/A — blocked
False Negatives	N/A — blocked
Signal Lag Ms	N/A — blocked

Historical replay is the gate that separates PROVISIONAL from VERIFIED. No plugin can pass this gate until compute() bodies are implemented. The Yahoo Finance provider is functional for OHLCV fetch and could supply historical bars once indicators exist; the Backtest Engine (currently a 14-line stub) would need to be implemented to drive replay.

7. Integration Verification

The verifier traced a single nominal request through the complete 10-stage pipeline: Market Data → Provider Router → Cache → Normalizer → Validation → Indicators → Pattern Recognition → Market Structure → Evidence/DNA → Dashboard. Each stage was checked for: (a) existence of the implementing component; (b) source line count; (c) presence of real methods (not just class declarations); (d) integration with the next stage.

Stage	Component	Exists	Notes
1.market_data	providers/yahoo/adapter.py	YES	Yahoo adapter implemented; fetches OHLCV via yfinance.
2.provider_router	providers/failover/	YES	Failover/router implemented.
3.cache	engines/data-engine/	YES	Data-engine has 2 files; 0 mention 'cache'. Engine itself is a stub (12 LoC).
4.normalizer	providers/yahoo/adapter.py (inline normalize)	NO	'normalize' referenced 0 times in Yahoo adapter.
5.validation	agents/validation/*/	YES	9 validators exist with real implementations.
6.indicators	plugins/indicators/*/	YES	14 indicator slots; 14 are stubs (all raise NotImplementedError).
7.patterns	plugins/patterns/*/	YES	6 pattern slots; 6 are stubs.
8.market_structur e	plugins/patterns/market-structure/	NO	No market-structure plugin exists; HH/HL/LH/LL/BOS/CHOCH/Liquidity Sweep detection is NOT implemented.
9.evidence_dna	engines/governance-engine/	YES	Governance engine has 6 Python files; audit_trail + self_healing present.
10.dashboard	apps/nextjs-dashboard/src/modules/	YES	12 dashboard modules: macro-intelligence, data-quality, self-validation, technical-analysis, dashboard, report-generator, market-intelligence, options-intelligence...

Pipeline broken at: stage 6 (indicators) — all indicator plugins are stubs. Stages 1, 2, 3, 5, 9, and 10 are real and functional; stage 4 (Normalizer) is implicit — Yahoo and Finnhub adapters perform response normalisation inline rather than via a dedicated normaliser module, which is acceptable but creates coupling. Stage 6 (Indicators) is where the pipeline breaks: all 14 indicators are stubs and cannot consume the validated OHLCV that stages 1–5 deliver. Stages 7 (Patterns) and 8 (Market Structure) are downstream of 6 and inherit the break. Stage 8 is doubly broken because no market-structure plugin exists at all.

7.1 End-to-End Latency (Stages 1–5 Only)

Latency was measured for stages 1–5 only because stages 6–10 are blocked. The Yahoo Finance adapter's typical round-trip for a 1-symbol 1-day OHLC fetch is approximately 400–800 ms (cold cache) and 1–5 ms (warm cache via failover cache). The failover router adds ~0.3 ms of dispatch overhead. Schema validation across 9 validators adds ~1.2 ms. The total Stage 1–5 latency budget is therefore approximately 5–10 ms warm and 400–800 ms cold — well within the 1-second refresh budget declared in plugin manifests. **Note:** these numbers come from Stage 16A Yahoo validation, not from this audit; the audit itself does not perform network calls.

8. Performance Verification

Performance was measured for the only operation available at audit time: module import + class instantiation. **26** plugins were sampled (those with source code). Max import time: **0.219 ms**. Average: **0.153 ms**. The compute() execution-time budget cannot be measured because no plugin currently computes. Note that import time is dominated by Python module-table overhead and is not representative of compute() performance; the figures below are upper-bound "first-call" overhead only.

8.1 Top 10 Slowest Plugin Imports

Rank	Plugin	Category	Import (ms)	Import OK
1	adx	indicators	0.219	YES
2	iv	options	0.174	YES
3	atr	indicators	0.166	YES
4	max-pain	options	0.164	YES
5	bollinger	indicators	0.161	YES
6	rsi	indicators	0.161	YES
7	sma	indicators	0.159	YES
8	stochastic	indicators	0.157	YES
9	macd	indicators	0.156	YES
10	ema	indicators	0.155	YES

Memory: not measured. The audit does not perform heap profiling. **CPU:** not measured. **Concurrency:** not measured — the engines' async/await usage has not been load-tested. **Scalability:** not measured — would require implementing backtest-engine first. **Resource leaks:** not measured — would require a long-running soak test against Yahoo adapter.

8.2 Performance Budget vs Actual

Metric	Budget	Actual	Status
Plugin compute() (warm)	≤ 5 ms	N/A (NotImplementedError)	BLOCKED
Plugin compute() (cold)	≤ 50 ms	N/A (NotImplementedError)	BLOCKED
Module import	≤ 100 ms	≤ 0.2 ms	PASS
Plugin instantiation	≤ 10 ms	≤ 1 ms (all plugins)	PASS
End-to-end pipeline	≤ 1000 ms	5–800 ms (stages 1–5 only)	PARTIAL
Memory per plugin	≤ 10 MB	Not measured	NOT EVALUATED
Concurrent plugins	200+	Not measured	NOT EVALUATED

9. Bugs Discovered

This section lists every defect identified during the audit, ranked by severity. Severity is assigned using the following rubric: **Critical** — blocks the platform's core declared function; **High** — blocks a major subsystem; **Medium** — creates wrong behavior or maintenance burden; **Low** — cosmetic or quality issue.

BUG-001 · **[Critical]** All 14 indicator plugins are stubs (NotImplementedError)

Evidence: `plugins/indicators/{ema,rsi,macd,sma,vwap,bollinger,adx,atr,cci,ichimoku,stochastic,wiliams-r,obv,fibonacci}/src/.../plugin.py:14 - `raise NotImplementedError("STEP 4 implementation")``

Reproduction: Instantiate any indicator plugin class (e.g., `EmaPlugin()`) and call `compute({}, {})`. Observe `NotImplementedError` raised before any computation.

Affected modules: Entire TA indicator layer (14 plugins). Pipeline stages 6+ cannot execute.

BUG-002 · **[Critical]** All 6 pattern plugins are stubs (NotImplementedError)

Evidence: `plugins/patterns/{candlestick,wyckoff,chan-theory,elliott-wave,smart-money,volume-profile}/src/.../plugin.py:9 - `raise NotImplementedError("STEP 4 implementation")``

Reproduction: Instantiate any pattern plugin class and call `compute({}, {})`. Observe `NotImplementedError`.

Affected modules: Entire pattern recognition layer (6 plugins). Wyckoff/Chan/Elliott/Smart Money/Candlestick/Volume Profile all unavailable.

BUG-003 · **[Critical]** Market Structure plugin does not exist

Evidence: `plugins/patterns/` has no `market-structure/` subdirectory. Filesystem check confirms absence.

Reproduction: `ls plugins/patterns/market-structure/` returns "No such file or directory".

Affected modules: No HH/HL/LH/LL/BOS/CHOCH/Liquidity Sweep detection. The foundational Stage 7 architectural layer is missing entirely.

BUG-004 · **[Critical]** 165 of 191 plugin slots have no source code

Evidence: 57/63 options plugins, 89/89 cross-market plugins, 10/10 news plugins, 9/9 forecast plugins have no `src/` directory.

Reproduction: For each affected plugin: `ls plugins/{category}/{name}/src/` returns "No such file or directory".

Affected modules: Entire options intelligence, cross-market, news, and forecast layers exist only as scaffolding metadata.

BUG-005 · [High] PluginManager loader looks for indicator.py, plugins use plugin.py

Evidence: engines/plugin-engine/src/athena_x_engine_plugin_engine/manager.py:125-128 –
``indicator_file = plugin_path / "src" / module_name / "indicator.py"```

Reproduction: Register a plugin, call `PluginManager.load("ema")`. Observe `ImportError "Cannot find indicator.py for plugin ema"` if package is not pip-installed.

Affected modules: All 14 indicator plugins + 6 pattern plugins cannot be loaded via `PluginManager` without pip install + the "any class" fallback.

BUG-006 · [High] PluginManager loader searches for wrong class names

Evidence: engines/plugin-engine/src/athena_x_engine_plugin_engine/manager.py:140-145 – searches ``EmaIndicator`, `EmaAgent`, `EMAIndicator`, `Indicator``. Actual class is ``EmaPlugin``.

Reproduction: After fixing BUG-005 (rename `indicator.py` → `plugin.py`), call `load("ema")`. Loader falls through all 4 candidates and uses "any class" fallback.

Affected modules: Same as BUG-005. Fragile: any plugin that legitimately defines multiple classes will have an arbitrary one selected.

BUG-007 · [High] Plugin compute() signature violates base TechnicalIndicator Protocol

Evidence: plugins/indicators/_base/src/athena_x_plugin_indicator_base/protocol.py:75-89 declares `compute(input_data: IndicatorInput, params: IndicatorParams) -> IndicatorOutput`.
plugins/indicators/ema/src/.../plugin.py:13 implements `compute(inputs: dict, params: dict) -> dict`.

Reproduction: Try to call ``ema.compute(IndicatorInput(...), IndicatorParams(...))`` from a caller expecting Protocol compliance. `TypeError` on the dict param.

Affected modules: Protocol contract is unenforceable. Even after implementations land, callers expecting `IndicatorOutput` receive raw dicts.

BUG-008 · [High] 6 of 14 engines are scaffolding-only (12 LoC stubs)

Evidence: engines/{ai-runtime,backtest-engine,data-engine,learning-engine,onnx-runtime,report-engine}/src/.../engine.py – each is 12 lines containing only ``class XxxEngine: "Implementation comes in STEP 4."``

Reproduction: Import any of these engines and call any method. `AttributeError` — class has no methods.

Affected modules: Backtesting, GPU model serving, ONNX browser runtime, multi-format report generation, and self-correction learning are all unavailable.

BUG-009 · [High] 11 of 16 providers are scaffolding-only

Evidence: providers/{alphavantage,cnbc,databento,flashalpha,fred,polygon,polymarket,reuters,sec,trading-economics,wsj}/src/.../adapter.py — each is ~26 LoC with class declaration only.

Reproduction: Instantiate any of these adapter classes. Object has no `fetch_*` methods.

Affected modules: Only 5/16 declared market data sources are reachable. Polygon, databento, FRED, SEC, and others required for institutional coverage are unavailable.

BUG-010 · [Medium] Plugin tests directory is empty across all 191 plugins

Evidence: For every plugin, tests/test_plugin.py contains only `"""Tests for X plugin."""` (1 line, 0 assertions).

Reproduction: Run `pytest plugins/` — collection finds 191 test files, all of which collect 0 test cases.

Affected modules: No regression protection for any plugin. CI test command "turbo run test" passes vacuously.

BUG-011 · [Medium] Duplicate plugin slots inflate count

Evidence: plugins/options/gamma-exposure/ AND plugins/options/gex/ declare the same metric. plugins/options/volatility-surface/ AND plugins/options/vol_surface/ are duplicates.

Reproduction: Compare manifest.yaml of gamma-exposure vs gex — both declare id "options.gamma-exposure" pattern.

Affected modules: Maintenance burden. When implementations land, two parallel implementations of the same metric will diverge.

BUG-012 · [Medium] plugin-engine has both real implementation files and a stub engine.py entrypoint

Evidence: engines/plugin-engine/src/athena_x_engine_plugin_engine/engine.py:7 — "Plugin Loader & Lifecycle. Implementation comes in STEP 4." But registry.py (120 LoC), manager.py (217 LoC), executor.py (125 LoC) all have real implementations.

Reproduction: Import `athena_x_engine_plugin_engine.engine` and inspect `PluginEngineEngine` class. Empty. But import `.manager` and `PluginManager` works.

Affected modules: Confusion about which file is the engine entrypoint. If a caller imports the engine module expecting the implementation, they get an empty class.

BUG-013 · [Low] Manifest YAML and Manifest Python dataclasses declare inconsistent plugin IDs

Evidence: plugins/indicators/ema/manifest.yaml declares id: "ema". plugins/indicators/ema/src/.../manifest.py declares id: "indicators.ema". Two different identifiers for the same plugin.

Reproduction: Compare the two files. Same for all 14 indicators and 6 patterns.

Affected modules: PluginRegistry may register the same plugin under two different IDs depending on which manifest is read first.

BUG-014 · [Low] Documentation claims "172 plugins implemented" — actual count is 0 functional

Evidence: Project Brain v1.0 summary states "172 plugins: TA indicators(14) + options(58) + cross-market(81) + news(10) + forecast(9)". Verification finds 0 functional plugins and 26 scaffolding stubs.

Reproduction: Cross-reference Project Brain v1.0 with the file-system audit. Every "implemented" plugin raises `NotImplementedError`.

Affected modules: Onboarding deception. New engineers will assume the platform works and waste time debugging nonexistent implementations.

10. Root Cause Analysis

The defects documented in Section 9 are not independent. They share three root causes that, if addressed, would resolve most of the surface-level bugs simultaneously.

10.1 Root Cause #1 — Scaffold-First Development Without Implementation Phase

The repository was built using a "scaffold-first" methodology: every plugin slot was created with `manifest.yaml`, `pyproject.toml`, `README.md`, an empty `tests` file, and a placeholder `plugin.py` containing ``raise NotImplementedError("STEP 4 implementation")``. The intent was apparently to establish the plugin architecture and contracts in Stages 5–6, then fill in implementations in "STEP 4" (likely a future sprint). **STEP 4 never arrived.** The scaffold-first phase completed (Stages 7–15 in the worklog all reference "172 plugins" as if they exist), but the implementation phase that would replace `NotImplementedError` with real `compute()` bodies was skipped or deferred indefinitely. This single root cause explains BUG-001, BUG-002, BUG-004, BUG-008, BUG-009, and BUG-010.

10.2 Root Cause #2 — Contract Drift Between Loader, Protocol, and Plugin Templates

Three independent files define what a "plugin" is: (a) `plugins/indicators/_base/protocol.py` declares the `TechnicalIndicator` Protocol with `compute(input_data: IndicatorInput, params: IndicatorParams) → IndicatorOutput`; (b) `engines/plugin-engine/manager.py` expects files named `indicator.py` with classes named `EmaIndicator` / `EmaAgent`; (c) the actual plugin template (used 20 times across indicators + patterns) creates `plugin.py` with class `EmaPlugin` implementing `compute(inputs: dict, params: dict) → dict`. None of these three definitions agree. The drift was not caught because no integration test ever exercised the path from `PluginManager` through to a real plugin invocation — there was never a working plugin to test against. This root cause explains BUG-005, BUG-006, BUG-007, and BUG-012.

10.3 Root Cause #3 — Documentation Tracking Slot Count Instead of Function Count

The Project Brain v1.0 documentation (27 documents, ~23,000 words) tracks progress as "172 plugins implemented", "58 options plugins", "81 cross-market plugins". These numbers count filesystem slots, not functional code. The result is a misleading positive feedback loop: each stage's worklog entry claims success based on slot count, the next stage assumes the previous stage's plugins are real, and the audit at Stage 16.1 reveals that none of them ever were. The certification reports in Stage 14.5 (Production Certification, PDF certificate) and Stage 15.7 (Architecture Freeze v1.0.0-rc1, 79/79 checks passed) were passed because the checks themselves were structural (`manifest` exists, file exists, lint passes) rather than functional (`compute` returns expected output). This root cause explains BUG-014 and is the reason the audit was commissioned.

10.4 Causal Chain

Scaffold-first methodology (Root Cause #1) created 191 slot directories with placeholder `compute()` bodies. **Contract drift** (Root Cause #2) went undetected because no plugin had a real `compute()` body to test against the contract. **Slot-count-based documentation** (Root Cause #3) made each successive stage believe the platform was further along than it was, allowing the gap to compound across 15+ stages of work. The architectural foundation (event bus, plugin-engine, providers, validators, governance,

narrative, forecast, trade, validation framework) is real; the plugin implementations that would sit on top of that foundation and produce actual trading intelligence are absent.

11. Recommended Fixes

Recommended fixes are listed in dependency order. Fixes higher in the list unblock fixes lower in the list. No fix should be attempted until its prerequisites are complete. **No fix involves rewriting working code** — every fix below either fills in a stub or corrects a contract mismatch.

ID	Severity	Title	Action	Unblocks / Prereq.
FIX-01	Critical	Reconcile the plugin contract	Choose ONE contract for plugins: either (a) keep Protocol with IndicatorInput/IndicatorOutput and update every plugin template to match, or (b) keep dict-based compute and update Protocol to match. Recommended: (a) because Protocol enforces type safety. Files to edit: plugins/indicators/_base/protocol.py (no change), every plugin template plugin.py (rewrite signature), plugin-engine executor.py (call with IndicatorInput, expect IndicatorOutput).	Unblocks: FIX-02 through FIX-06.
FIX-02	Critical	Reconcile PluginManager loader with plugin filename + class name	Update engines/plugin-engine/manager.py lines 125-128 to look for plugin.py (in addition to indicator.py), and lines 140-145 to look for {PluginId.title()}Plugin and {PluginId.upper()}Plugin class names. Add a unit test that loads every shipped plugin.	Unblocks: every plugin can be loaded. Prerequisite: FIX-01.
FIX-03	Critical	Implement the 14 indicator plugins	For each indicator (EMA, RSI, MACD, SMA, VWAP, Bollinger, ADX, ATR, CCI, Ichimoku, Stochastic, Williams %R, OBV, Fibonacci): replace `raise NotImplementedError` with a real algorithm. Use numpy for vectorised math. Add 5+ test cases per indicator with known historical inputs and expected outputs (the test corpus from Section 4).	Unblocks: trading-logic verification, historical replay, downstream patterns. Prerequisite: FIX-01, FIX-02.
FIX-04	Critical	Implement the 6 pattern plugins	For each pattern (Candlestick, Wyckoff, Chan Theory, Elliott Wave, Smart Money, Volume Profile): replace NotImplementedError with real algorithm. Each pattern depends on indicators from FIX-03 — wire those dependencies in manifest.yaml.	Unblocks: pattern layer. Prerequisite: FIX-03.
FIX-05	Critical	Create the Market Structure plugin	Add plugins/patterns/market-structure/ with manifest.yaml + plugin.py implementing HH/HL/LH/LL detection, BOS, CHOCH, and Liquidity Sweep. This is the foundational layer for Wyckoff, Smart Money, and Chan Theory — without it, those patterns have no input.	Unblocks: meaningful pattern layer. Prerequisite: FIX-03.
FIX-06	Critical	Decide policy for the 165 sourceless plugin slots	Either (a) implement them in priority order (ODTE options first, then cross-market correlations, then news classifiers, then forecast models), or (b) delete the slots and recreate them as they are implemented. Recommended: (a) with explicit per-stage tracking. Mark each as "not yet implemented" in the registry rather than letting the slot imply functionality.	Unblocks: realistic progress tracking. Prerequisite: FIX-01.
FIX-07	High	Implement the 6 stub engines	data-engine (cache + normalizer), ai-runtime (GPU model server), backtest-engine (vectorbt), learning-engine (self-correction), onnx-runtime (browser model runtime), report-engine (multi-format PDF/HTML/Markdown). Each is currently 12 LoC.	Unblocks: historical replay (backtest-engine), real report generation (report-engine). Prerequisite: FIX-03.
FIX-08	High	Implement the 11 stub providers	In priority order: polygon (institutional OHLCV + options), databento (futures + options), FRED (macroeconomic series), SEC (filings), then the rest. Each is currently ~26 LoC.	Unblocks: institutional data coverage beyond Yahoo. Prerequisite: FIX-02.
FIX-09	Medium	Add real tests for every plugin	Replace `"""Tests for X plugin."""` with at least 5 test functions per plugin covering: (a) happy-path compute; (b) empty input; (c) insufficient warmup; (d) NaN/Inf handling; (e) parameter validation. Use the test corpus from Section 4.	Unblocks: regression protection. Prerequisite: FIX-03.
FIX-10	Medium	Remove duplicate plugin slots	Audit and merge: gamma-exposure ↔ gex; volatility-surface ↔ vol_surface; vix_confirming ↔ vix_not_confirming (consider one plugin with a "mode" param); spy_es_corr ↔ spy_es_divergence; qqz_leads_spy ↔ nvda_leads_qqz; us2y ↔ us30y ↔ tnx (one treasury-yield plugin with "tenor" param).	Unblocks: maintainable plugin count. Prerequisite: FIX-06.

FIX-11	Medium	Reconcile manifest.yaml and manifest.py IDs	For every plugin that has both, ensure the id field matches exactly. Either delete manifest.py (preferred — single source of truth in manifest.yaml) or delete manifest.yaml (if the Python dataclass is needed for type-safe access). Recommended: delete manifest.py and have the registry load manifest.yaml only.	Unblocks: stable registry IDs. Prerequisite: none.
FIX-12	Low	Update Project Brain v1.0 documentation	Replace "172 plugins implemented" with "191 plugin slots declared; X implemented; Y scaffolding; Z not yet started". Track implementation count, not slot count. Each stage's worklog entry should report (slots declared, slots with code, slots with passing tests).	Unblocks: accurate onboarding. Prerequisite: FIX-06.

12. Certification Status

Every plugin, engine, and provider receives one of three certification statuses:

- **VERIFIED** — passes functional, logic, integration, performance, AND historical validation. Required for production trading.
- **PROVISIONAL** — functionally correct but lacking full validation evidence. Suitable for staging / paper trading.
- **FAILED** — logic bug, incorrect calculations, integration failure, performance issue, or missing dependency. Not suitable for any use.

12.1 Aggregate Certification

Layer	VERIFIED	PROVISIONAL	FAILED	Total
Plugins	0	0	191	191
Engines	0	8	6	14
Providers	0	5	11	16
TOTAL	0	13	208	221

12.2 Plugin-Layer Certification by Category

Category	VERIFIED	PROVISIONAL	FAILED	Total	Failure Rate
cross-market	0	0	89	89	100%
forecast	0	0	9	9	100%
indicators	0	0	14	14	100%
news	0	0	10	10	100%
options	0	0	63	63	100%
patterns	0	0	6	6	100%

The plugin layer has a **100% failure rate** across all six categories. This is not a quality-of-implementation issue; it is a fundamental absence-of-implementation issue. No plugin in any category can be trusted to produce a correct output, because no plugin in any category produces any output at all.

12.3 Engine-Layer Certification

Engine	LoC	Stub	Cert	Reason
ai-runtime	14	YES	FAILED	engine is scaffolding-only (14 LoC; entrypoint class declared with no body)
backtest-engine	14	YES	FAILED	engine is scaffolding-only (14 LoC; entrypoint class declared with no body)
cross-market-plugin-engine	293	no	PROVISIONAL	real implementation (293 LoC); needs production burn-in

data-engine	14	YES	FAILED	engine is scaffolding-only (14 LoC; endpoint class declared with no body)
forecast-engine	476	no	PROVISIONAL	real implementation (476 LoC); needs production burn-in
governance-engine	580	no	PROVISIONAL	real implementation (580 LoC); needs production burn-in
learning-engine	14	YES	FAILED	engine is scaffolding-only (14 LoC; endpoint class declared with no body)
narrative-engine	552	no	PROVISIONAL	real implementation (552 LoC); needs production burn-in
onnx-runtime	14	YES	FAILED	engine is scaffolding-only (14 LoC; endpoint class declared with no body)
options-plugin-engine	181	no	PROVISIONAL	real implementation (181 LoC); needs production burn-in
plugin-engine	1010	YES	PROVISIONAL	real implementation (1010 LoC) but contains at least one stub endpoint file: engines/plugin-eng...
report-engine	14	YES	FAILED	engine is scaffolding-only (14 LoC; endpoint class declared with no body)
trade-engine	542	no	PROVISIONAL	real implementation (542 LoC); needs production burn-in
validation-framework	346	no	PROVISIONAL	real implementation (346 LoC); needs production burn-in

The 8 PROVISIONAL engines represent the platform's real institutional logic and should be prioritised for VERIFIED certification once their dependent plugins (FIX-03 through FIX-06) are implemented. The 6 FAILED engines must be implemented (FIX-07) before they can be certified.

12.4 Provider-Layer Certification

Provider	LoC	Adapter Class	Tests	Cert
alphavantage	27	AlphavantageAdapter	empty	FAILED
cnbc	26	CnbcAdapter	empty	FAILED
cnn	126	CNNAdapter	yes (77L)	PROVISIONAL
databento	26	DatabentoAdapter	empty	FAILED
failover	123	—	yes (104L)	PROVISIONAL
finnhub	143	FinnhubAdapter	yes (58L)	PROVISIONAL
flashalpha	27	FlashalphaAdapter	empty	FAILED
fred	27	FredAdapter	empty	FAILED
polygon	27	PolygonAdapter	empty	FAILED
polymarket	26	PolymarketAdapter	empty	FAILED
reuters	26	ReutersAdapter	empty	FAILED
sec	26	SecAdapter	empty	FAILED
simulated	122	SimulatedAdapter	yes (62L)	PROVISIONAL
trading-economics	26	TradingEconomicsAdapter	empty	FAILED
wsj	26	WsjAdapter	empty	FAILED
yahoo	176	YahooAdapter	yes (121L)	PROVISIONAL

12.5 Final Certification Verdict

PLATFORM CERTIFICATION: FAILED

The ATHENA-X platform as a whole is FAILED at Stage 16.1. Of 221 declared components (191 plugins + 14 engines + 16 providers), 0 are VERIFIED, 13 are PROVISIONAL (8 real engines + 5 real providers), and 208 are FAILED. The platform's architectural foundation (event bus, plugin registry, governance engine, narrative engine, validation framework, Yahoo/CNN/Finnhub/Simulated/Failover adapters) is real and could support institutional trading — but the plugin implementations that would produce trading intelligence are absent. The platform cannot generate a single trading signal at this time. Recommended next stage: implement FIX-01 through FIX-05 (reconcile contracts + implement 14 indicators + 6 patterns + market structure) before any further platform-level certification is attempted.

Report generated by: Stage 16.1 Plugin Discovery & Verification Master audit script

Evidence file: /home/z/my-project/scripts/stage16_1_evidence.json

Verification script: /home/z/my-project/scripts/stage16_1_verify_plugins.py

Audit date: 2026-07-19

Audit scope: non-destructive static + dynamic analysis; no source code modified